

Python Code	Explanation
Bar Chart	
import pandas as pd import matplotlib.pyplot as plt import seaborn as sns	These Lines of code upload packages available in Python. Pandas for the analyzation of data, matplotlib for plots, and seaborn for advanced plots.
nhts_data = pd.read_csv('NHTS.csv')	This line uploads a The NHTS data set into Python.
trip_freq = (nhts_data.groupby('household_id')['travel_day'].count().reset_index(n ame='trip_frequency'))	This line computes the trip frequency by using the travel_day column. It then adds it to a clean table.
trip_freq = trip_freq.merge(nhts_data[['household_id', 'household_income']], drop_duplicates(), on='household_id', how='left')	This line of code is adding the household income to the clean dataset.
avg_trips = (trip_freq.groupby('household_income')['trip_frequency'].mean().drop na()) income_order = ["I don't know", 'I prefer not to answer', 'Less than \$10,000', '\$10,000 to \$14,999', '\$15,000 to \$24,999', '\$25,000 to \$34,999', '\$35,000 to \$49,999', '\$50,000 to \$74,999', '\$75,000 to \$99,999',	This chunk of code groups households by their incomes, then averaged their number of trips.

'\$100,000 to \$124,999', '\$125,000 to \$149,999', '\$150,000 to \$174,999', '\$175,000 or more']	
avg_trips = avg_trips.reindex(income_order)	This code reorders the dataset to match the order of income_order
plt.figure(figsize=(12,6)) avg_trips.plot(kind='bar', color=sns.color_palette('plasma', len(avg_trips)), edgecolor='hotpink') plt.xlabel('Household Income') plt.ylabel('Average Trips per Household') plt.title('Average Trip Frequency by Household Income') plt.xticks(rotation=45) plt.tight_layout() plt.show()	This chunk of code creates a bargraph based on the clean dataset. The first line sets the size of the graph. The next defines the type of graph, color pallete. The next few add titles to the x axis, y axis, and the title. Then the lines following print the graph constructed.
Two Time Series	
import pandas as pd import matplotlib.pyplot as plt import seaborn as sns	These Lines of code upload packages available in Python. Pandas for the analyzation of data, matplotlib for plots, and seaborn for advanced plots.

<code>sns.set_style('whitegrid')</code>	This sets the style of the plot, in this case is a grid.
<code>ngsim_data = pd.read_csv('NGSIM.csv')</code>	This code uploads the NGSIM data set to python.
<code>traj_id = 1</code> <code>data_subset = ngsim_data[ngsim_data['trajectory_number'] == traj_id]</code>	This code specifies what Trajectory number is being worked with. It then filters the data to match the traj_id.
<code>plt.figure(figsize=(12, 5))</code> <code>plt.plot(data_subset['Time'], data_subset['leader_speed(m/s)'], label='Leader Speed', color='steelblue', linestyle='--')</code> <code>plt.plot(data_subset['Time'], data_subset['follower_speed(m/s)'], label='Follower Speed', color='coral', linewidth=2)</code>	This chunk of code creates line graph comparing the two sets of data. It first decides the size pf the graph then it assigns a color and pattern to each of the lines.
<code>plt.title(f'Time-Series: Speed Comparison (Trajectory {traj_id})', fontsize=14)</code> <code>plt.xlabel('Time (seconds)', fontsize=12)</code> <code>plt.ylabel('Speed (m/s)', fontsize=12)</code> <code>plt.legend()</code> <code>plt.tight_layout()</code> <code>plt.show()</code>	The lines of code add a custom title. It also adds titles to the x axis and y axis. It also adds a legend or distinction. Then the lines following print the graph constructed.
<code>plt.figure(figsize=(12, 5))</code>	This chunk of code creates

<pre>sns.lineplot(x=data_subset['Time'], y=data_subset['follower_acc(m/s^2)'], color='darkred', linewidth=1.5)</pre>	line plot of the data. It first decides the size of the graph then it assigns the x and y axis, and the characteristics of the line
<pre>plt.axhline(0, color='black', linewidth=1, alpha=0.5)</pre>	This adds a horizontal line on the graph for better analyzation.
<pre>plt.title(f'Time-Series: Follower Acceleration Profile (Trajectory {traj_id}'), fontsize=14) plt.xlabel('Time (seconds)', fontsize=12) plt.ylabel('Acceleration (m/s²)', fontsize=12) plt.tight_layout() plt.show()</pre>	The lines of code add a custom title. It also adds titles to the x axis and y axis. It also adds a legend or distinction. Then the lines following print the graph constructed.
IDM Simulation	
<pre>import pandas as pd import numpy as np import matplotlib.pyplot as plt</pre>	These Lines of code upload packages available in Python. Pandas for the analyzation of data, matplotlib for plots, and seaborn for advanced plots.
<pre>df = pd.read_csv("NGSIM.csv")</pre>	This code uploads the NGSIM data set to python.

<pre> traj_id = df["trajectory_number"].iloc[0] traj_df = df[df["trajectory_number"] == traj_id].copy() traj_df = traj_df.sort_values("Time").reset_index(drop=True) </pre>	This extract data that based on the trajectory number.
<pre> time = traj_df["Time"].values leader_position = traj_df["leader_position(m)"].values leader_speed = traj_df["leader_speed(m/s)"].values leader_acc = traj_df["leader_acc(m/s^2)"].values follower_position = traj_df["follower_position(m)"].values follower_speed = traj_df["follower_speed(m/s)"].values follower_acc = traj_df["follower_acc(m/s^2)"].values </pre>	These chunk of codes converts the data to arrays for better usage of the plot. It does this for the following values.
<pre> dt = time[1] - time[0] n_steps = len(time) </pre>	This code sets the time interval between each measurement. The next sets the length of measurements to equal the time interval.
<pre> def idm_acceleration(v, s, delta_v, v0, s0, T, a, b, delta): s = max(s, 1e-6) s_star = s0 + v*T + (v*delta_v)/(2*np.sqrt(a*b)) s_star = max(s_star, s0) return a * (1 - (v/v0)**delta - (s_star/s)**2) </pre>	This code defines a function that gives the acceleration of IDM.
<pre> v0 = 30 s0 = 2 T = 1.5 a_param = 1.0 b = 1.5 delta = 4 </pre>	The variables are set for the functions
<pre> sim_position = np.zeros(n_steps) sim_speed = np.zeros(n_steps) sim_acc = np.zeros(n_steps) sim_position[0] = follower_position[0] sim_speed[0] = follower_speed[0] </pre>	This code set up an array to store the follower's information

<pre> for i in range(n_steps - 1): s = leader_position[i] - sim_position[i] delta_v = sim_speed[i] - leader_speed[i] acc = idm_acceleration(sim_speed[i], s, delta_v, v0, s0, T, a_param, b, delta) sim_acc[i] = acc sim_speed[i+1] = sim_speed[i] + acc * dt sim_position[i+1] = sim_position[i] + sim_speed[i] * dt </pre>	This code updates the position and calculated how the follower would move according to idm_acceleration.
<pre> plt.figure(figsize=(10, 6)) plt.plot(time, leader_acc, label="Leader Acceleration") plt.plot(time, follower_acc, label="Real Follower Acceleration") plt.plot(time, sim_acc, label="IDM Simulated Acceleration") plt.xlabel("Time") plt.ylabel("Acceleration (m/s^2)") plt.title("IDM Simulation vs Real Data") plt.legend() plt.grid(True) plt.show() </pre>	This chunk of code creates the plot base on the information. The first line determines the size of the entire plot. The next lines set information that will be included with the labels according to the data. Next lines set the title for the x/y axis and title. It then adds a legend.
Histogram	
<pre> import pandas as pd import matplotlib.pyplot as plt </pre>	These Lines of code upload packages available in Python. Pandas for the analyzation of data and matplotlib for plots.

<code>df = pd.read_csv("NGSIM.csv")</code>	This code uploads the NGSIM data set to python.
<code>variable = "follower_speed(m/s)"</code>	Assigns a string to variable
<pre>plt.figure(figsize=(8, 6)) plt.hist(df[variable].dropna(), bins=30) plt.xlabel(variable) plt.ylabel("Frequency") plt.title(f"Distribution of {variable}") plt.grid(True) plt.show()</pre>	This create the graph, first size is chosen. The second line creates a histogram of the data chosen to be variable. Whish is follower_speed(m/s). The next lines assign titles to the x and y axis and adds a title. The next line confirms grid and then displays the graph created.